Exercício prático: Sistema de Controle de Notas

Nível intermediário
Individual

Objetivo: verificar se o aluno sabe utilizar for, while e do while, compreendendo quando cada estrutura é mais adequada.

Enunciado para os alunos

Você foi contratado para desenvolver um pequeno sistema que recebe notas de alunos, calcula resultados e permite que o usuário interaja com um menu. O programa deverá realizar as tarefas abaixo.

Parte 1 — Cadastro de notas
O programa deve solicitar as notas de 5 alunos.

Utilize uma estrutura de repetição para receber as cinco notas.

Calcule a soma e a média das notas.

Ao final, mostre a média da turma.

```c
#include<stdio.h>

int i;
float nota, soma = 0, media;

int main(){


    for(i = 1; i <=5; i++){
        printf("Digite a nota do aluno %d: ", i);
        scanf("%f", &nota);
        soma = soma + nota;
    }
    media = soma / 5;

    printf("A media da turma e: %.2f\n", media);

    return 0;
    }

```

Perguntas para responder:

Por que a estrutura escolhida é adequada para esse problema?

R: Pois sabemos exatamente quantas vezes o loop precisa rodar (5 alunos). O for é ótimo para situações com quantidade fixa de repetição.

O que aconteceria se a repetição fosse executada apenas quatro vezes?

R: O programa só recolheria a nota de 4 alunos e deixaria o último de fora, ocasionando com que a turma ficasse com a média final incorreta.

Parte 2 — Senha de acesso
Crie uma etapa de autenticação para o sistema.

A senha correta é 1234.

Solicite a senha ao usuário.

Enquanto a senha estiver errada, solicite novamente.

Quando a senha estiver correta, mostre a mensagem Acesso permitido!.

```c
#include<stdio.h>

int senha;

int main(){

    printf("Digite a senha: ");
    scanf("%d", &senha);

    while (senha != 1234){
        printf("Senha incorreta! Digite novamente: ");
        scanf("%d", &senha);
    }

    printf("Acesso permitido!\n");

    return 0;
}
```

Perguntas para responder:

O que acontece se o usuário acertar a senha na primeira tentativa?

R: O loop do while é ignorado e o programa é encerrado.

Por que essa situação pode ser resolvida com while?

R: Porque o número de tentativas é incerto (não sabemos quantas vezes o usuário vai errar).

Qual erro poderia fazer o programa entrar em um laço infinito?

R: Esquecer de atualizar o valor da variável senha dentro do loop (esquecer de colocar um novo scanf lá dentro). Se o programa não ler a nova tentativa, a condição nunca mudará e o loop vai rodar pra sempre.

Parte 3 — Menu interativo
Depois de entrar no sistema, o usuário deverá visualizar este menu:

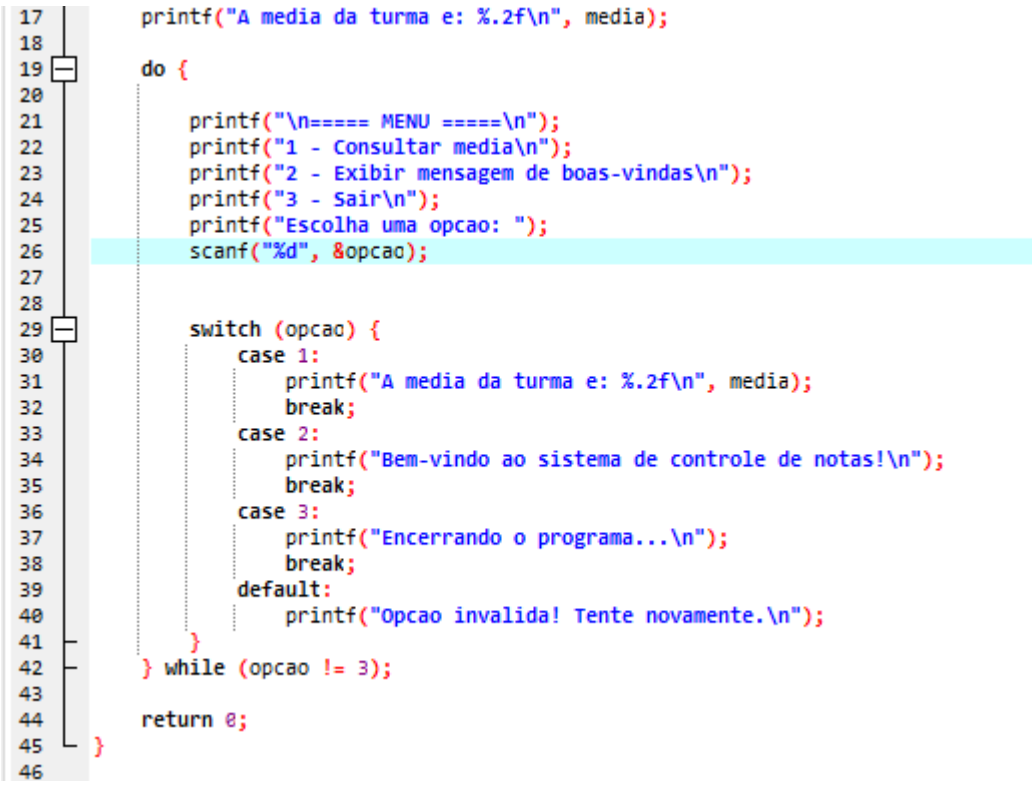
text
===== MENU =====
1 - Consultar média
2 - Exibir mensagem de boas-vindas
3 - Sair
Escolha uma opção:
O menu deverá aparecer pelo menos uma vez.

Se o usuário escolher 1, mostre a média calculada na Parte 1.

Se escolher 2, exiba uma mensagem de boas-vindas.

Se escolher 3, encerre o programa.

Se escolher uma opção inválida, informe o erro e exiba o menu novamente.



```c
    printf("A media da turma e: %.2f\n", media);

    do {

        printf("\n===== MENU =====\n");
        printf("1 - Consultar media\n");
        printf("2 - Exibir mensagem de boas-vindas\n");
        printf("3 - Sair\n");
        printf("Escolha uma opcao: ");
        scanf("%d", &opcao);


        switch (opcao) {
            case 1:
                printf("A media da turma e: %.2f\n", media);
                break;
            case 2:
                printf("Bem-vindo ao sistema de controle de notas!\n");
                break;
            case 3:
                printf("Encerrando o programa...\n");
                break;
            default:
                printf("Opcao invalida! Tente novamente.\n");
        }
    } while (opcao != 3);

    return 0;
}

```

Perguntas para responder:

Por que do while combina com esse problema?

R: Porque o menu precisa aparecer na tela pelo menos uma vez para a pessoa poder escolher o que quer fazer.

O que diferencia seu funcionamento do while?

R: O do-while faz o código rodar primeiro e só testa a regra no final (roda no mínimo uma vez). O while testa a regra no início, então pode nem rodar se a condição já for falsa.

O que acontece se o usuário digitar 3 logo na primeira exibição?

R: O programa mostra a mensagem de encerramento e fecha na hora, sem repetir o menu.

Parte 4 - Analise o código abaixo, sem executá-lo inicialmente:

c

```
int i = 1;

while (i <= 5) {
    printf("%d ", i);
    i++;
}
```

Responda:

Qual será a saída do programa?

R: 12345

Quantas vezes o laço será executado?

R: O laço será executado 5 vezes (quando i for: 1, 2, 3, 4 e 5).

O que aconteceria se retirássemos i++?

R: O valor de i ficaria preso em 1 para sempre, o que faria a condição i <= 5 ser sempre verdadeira. Isso faria o programa entrar em um laço infinito, imprimindo o número 1 sem parar até o programa ser forçado a fechar.

Reescreva o código usando for.

```c
#include <stdio.h>

int i;

int main() {

    for (i = 1; i <= 5; i++) {
        printf("%d ", i);
    }


    return 0;
}
```

Reescreva o código usando do while.

```c
#include <stdio.h>

int i = 1;

int main() {

    do {
        printf("%d ", i);
        i++;
    } while (i <= 5);

    return 0;
}
```